

Lecture 4: Advanced Pattern Printing Paradigms

This module transitions from basic structural loops to complex, multi-phase coordinate mapping. To conquer advanced patterns, you must visualize the console not as a text editor, but as a strict X/Y grid (Rows and Columns) where mathematical relationships dictate exactly what character gets printed at any specific coordinate.

Problem 1: Continuous Alphanumeric Triangle

1. Problem Statement & Constraints

Generate a right-angled triangle mapping the sequential English alphabet. The character must increment continuously across rows and columns without resetting.

Expected Output (for $N = 4$):

Plaintext

```
A
B C
D E F
G H I J
```

2. Core Intuition

- **Coordinate Independence:** Unlike basic patterns where the value printed depends on the row (i) or column (j), this pattern relies on a completely independent state variable.
- **ASCII Incrementing:** Characters in C++ are stored as integers under the hood (ASCII values). We can initialize a tracking variable to 'A' and mathematically increment it ($ch++$) just like an integer.

3. Algorithmic Steps

1. Initialize the grid boundary N .
2. Initialize the outer loop iterator $i = 1$ and a global character tracker `char ch = 'A'`.
3. Open the outer `while` loop bound by $i \leq n$ to control the vertical rows.
4. Initialize the inner loop iterator $j = 1$.
5. Open the inner `while` loop bound by $j \leq i$ (since the number of columns in this triangle equals the current row number).
6. Print the current character `ch`, then immediately increment both `ch` and `j`.

7. Once the inner loop completes, print a newline (endl) and increment i .

4. Dry Run

Input: N = 3

- 1. i = 1 , ch = 'A' . Inner loop runs 1 time. Prints A . ch becomes 'B' . Newline.
- 2. i = 2 , ch = 'B' . Inner loop runs 2 times. Prints B , ch becomes 'C' . Prints C , ch becomes 'D' . Newline.
- 3. i = 3 , ch = 'D' . Inner loop runs 3 times. Prints D , ch becomes 'E' . Prints E , ch becomes 'F' . Prints F , ch becomes 'G' . Newline.

Output: Matches the expected structural progression.

5. Complexity Analysis Table

Metric	Complexity	Justification
Time Complexity	$O(N^2)$	The outer loop runs N times, and the inner loop runs an arithmetic progression ($1 + 2 + 3 \cdots + N$), which mathematically equates to $\frac{N(N+1)}{2}$.
Space Complexity	$O(1)$	No dynamic memory or auxiliary arrays are used. Variables are stored in constant space.

Code Implementation

C++

```
#include <iostream>
using namespace std;

int main() {
    int n = 4;
    int i = 1;
    char ch = 'A'; // Independent global state tracker

    while (i <= n) {
        int j = 1;
        while (j <= i) {
            cout << ch << " ";
            ch++; // Implicitly increments the ASCII value
            j++;
        }
        cout << endl;
        i++;
    }
}
```

```
    return 0;
}
```

Problem 2: Right-Aligned Star Triangle

1. Problem Statement & Constraints

Print a right-aligned triangle of stars where the base is completely solid, and the peak is aligned to the far right of the console.

Expected Output (for $N = 4$):

Plaintext

```
* * * * *
* * * * *
* * * * *
* * * * *
```

2. Core Intuition

- **Dual-Phase Rows:** You cannot simply command the terminal to "align right." The terminal prints strictly left-to-right. Therefore, every row consists of two distinct phases: printing invisible boundaries (Spaces) first, followed by the visible characters (Stars).
- **Space Formula:** In any given row i , the number of empty spaces required before printing stars is exactly $N - i$.
- **Star Formula:** The number of stars required is exactly equal to i .

3. Algorithmic Steps

1. Initialize the grid boundary N and the outer row tracker $i = 1$.
2. Open the outer `while` loop bound by $i \leq n$.
3. **Phase 1 (Spaces):** Initialize a variable `space = n - i`. Run a nested `while` loop as long as `space > 0`. Print a space and decrement the `space` counter.
4. **Phase 2 (Stars):** Initialize a variable `j = 1`. Run a second sequential nested `while` loop bound by `j <= i`. Print a star and increment `j`.
5. Drop to a newline using `endl` and increment i .

4. Dry Run

Input: $N = 3$

1. $i = 1$. Spaces = $3 - 1 = 2$. Prints . Stars = 1. Prints `*`. Newline.
2. $i = 2$. Spaces = $3 - 2 = 1$. Prints . Stars = 2. Prints `**`. Newline.
3. $i = 3$. Spaces = $3 - 3 = 0$. Prints nothing. Stars = 3. Prints `***`. Newline.

5. Complexity Analysis Table

Metric	Complexity	Justification
Time Complexity	$O(N^2)$	Despite having two inner loops, they run sequentially (not nested inside each other). For each row, the total prints (spaces + stars) equals N . Thus, $N \times N = N^2$.
Space Complexity	$O(1)$	Strictly utilizes scalar counter variables.

Code Implementation

```
#include <iostream>
using namespace std;

int main() {
    int n = 4;
    int i = 1;

    while (i <= n) {
        // Phase 1: Print required empty padding
        int space = n - i;
        while (space > 0) {
            cout << "  "; // Double space to match star width
            space--;
        }

        // Phase 2: Print visible stars
        int j = 1;
        while (j <= i) {
            cout << "* ";
            j++;
        }

        cout << endl;
        i++;
    }
    return 0;
}
```

Problem 3: The "Dabangg" Multi-Phase Pattern

1. Problem Statement & Constraints

Construct a complex mirrored geometric pattern consisting of ascending numbers, a void of stars, and descending numbers. This is a classic test of multi-loop logic synchronization.

Expected Output (for $N = 5$):

Plaintext

```

1 2 3 4 5 5 4 3 2 1
1 2 3 4 * * 4 3 2 1
1 2 3 * * * * 3 2 1
1 2 * * * * * 2 1
1 * * * * * * * 1

```

2. Core Intuition

- **Tri-Phase Row Execution:** This pattern is not one grid, but three distinct geometric shapes stitched horizontally together. Each row i requires three sequential loops.
- **Triangle 1 (Ascending Numbers):** An inverted triangle printing 1 to $N - i + 1$.
- **Triangle 2 (The Star Void):** A pyramid of stars expanding downwards. The number of stars follows the algebraic sequence $0, 2, 4, 6, 8$, which maps exactly to the formula $2 \times (i - 1)$.
- **Triangle 3 (Descending Numbers):** A mirrored inverted triangle printing backwards from $N - i + 1$ down to 1 .

3. Algorithmic Steps

1. Set boundary N and start the outer loop $i = 1$.
2. **First Inner Loop:** Run j from 1 up to $N - i + 1$. Print the value of j .
3. **Second Inner Loop:** Calculate the total star count as $2 \times (i - 1)$. Use a loop to print that exact quantity of stars.
4. **Third Inner Loop:** Initialize $k = N - i + 1$. Run the loop backwards ($k \geq 1$), printing k and decrementing it.
5. Print a newline, increment i , and loop until N rows are drawn.

4. Dry Run

Input: $N = 4$

Row 1 ($i=1$):

- **Loop 1:** $N - i + 1 = 4$. Prints $1\ 2\ 3\ 4$.
- **Loop 2:** $2 \times (1 - 1) = 0$. Prints zero stars.
- **Loop 3:** Starts at 4 , decrements to 1 . Prints $4\ 3\ 2\ 1$.

Row 2 ($i=2$):

- **Loop 1:** $N - i + 1 = 3$. Prints $1\ 2\ 3$.
- **Loop 2:** $2 \times (2 - 1) = 2$. Prints $*\ *$.
- **Loop 3:** Starts at 3 , decrements to 1 . Prints $3\ 2\ 1$.

5. Complexity Analysis Table

Algorithmic Phase	Target Matrix	Complexity
Ascending Numbers	Left Inverted Triangle	$O(N^2)$ Time
Star Void	Central Pyramid	$O(N^2)$ Time
Descending Numbers	Right Mirrored Inverted Triangle	$O(N^2)$ Time
Overall Execution	Full Consolidated Pattern	$O(N^2)$ Time, $O(1)$ Space

Code Implementation

C++

```
#include <iostream>
using namespace std;

int main() {
    int n = 5;
    int i = 1;

    while(i <= n) {

        // Phase 1: Ascending Number Block (1 to N-i+1)
        int j = 1;
        while(j <= n - i + 1) {
            cout << j << " ";
            j++;
        }

        // Phase 2: Expanding Star Void Block (2 * (i-1))
        int stars = 2 * (i - 1);
        while(stars > 0) {
            cout << "* ";
            stars--;
        }

        // Phase 3: Descending Number Block (N-i+1 down to 1)
        int k = n - i + 1;
        while(k >= 1) {
            cout << k << " ";
            k--;
        }

        // Finalize Row
        cout << endl;
        i++;
    }
}
```

```
    return 0;  
}
```